

1 Network Flow

1.1 Model, residual arcs, and augmentation

Let $G = (V, E)$ be a directed network with distinct source s , sink t , capacities $c_e \geq 0$, $n = |V|$, and $m = |E|$. Parallel and antiparallel arcs are allowed; each original arc has its own identity. A feasible flow satisfies

$$0 \leq f_e \leq c_e \quad (e \in E), \quad \sum_{e \in \delta^-(v)} f_e = \sum_{e \in \delta^+(v)} f_e \quad (v \neq s, t).$$

Here $\delta^+(S)$ and $\delta^-(S)$ are the original arcs leaving and entering S , respectively. The objective is the *net* source outflow

$$|f| = \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e.$$

It equals net inflow at t . If there are no arcs entering s , it is simply source outflow. Nonnegativity is part of feasibility.

Residual graph G_f . For each original arc $e = (u, v)$, keep two *labelled* residual arcs:

label and direction	residual capacity	augmentation by Δ
$e^+ : u \rightarrow v$	$c_e - f_e$	$f_e \leftarrow f_e + \Delta$
$e^- : v \rightarrow u$	f_e	$f_e \leftarrow f_e - \Delta$

Only positive-capacity residual arcs are available. In particular, when (u, v) and (v, u) are both original arcs, an arc directed $u \rightarrow v$ may mean increasing one original flow or decreasing the other. Its label, not membership of (u, v) in E , determines the update. For a simple residual s - t path P , set $\Delta = \min_{a \in P} c_f(a)$ and apply its labelled updates. Bounds $0 \leq f_e \leq c_e$ remain valid; every internal path vertex has zero change in net outflow; $|f|$ increases by Δ .

Why reverse arcs are necessary. Take unit-capacity arcs $s \rightarrow a, s \rightarrow b, a \rightarrow b, a \rightarrow t, b \rightarrow t$. After sending one unit along $s \rightarrow a \rightarrow b \rightarrow t$, no forward-only augmenting path remains, although the optimum is 2. The residual path $s \rightarrow b \rightarrow a \rightarrow t$ uses the reverse of $a \rightarrow b$, cancels that unit, and produces the two direct unit routes $s \rightarrow a \rightarrow t$ and $s \rightarrow b \rightarrow t$.

Comparing two flows: use the residual network. For feasible original flows f, g , set $h(e^+) = \max\{g_e - f_e, 0\}$ and $h(e^-) = \max\{f_e - g_e, 0\}$. These values respect residual capacities; conservation follows by subtracting the two original conservation equations, and $|h| = |g| - |f|$. Conversely, a residual flow h updates the original flow by $f'_e = f_e + h(e^+) - h(e^-)$, with $0 \leq f'_e \leq c_e$ and $|f'| = |f| + |h|$. Thus residual flow models possible improvement. The ordinary vector $g - f$ need not be a nonnegative flow in G : an arc may carry less under g than under f . Its negative component belongs on the corresponding reverse residual arc.

1.2 Cut identities and max-flow/min-cut

An s - t cut is a partition $(S, V \setminus S)$ with $s \in S, t \notin S$; its capacity counts only arcs directed *out of* S : $c(S) = \sum_{e \in \delta^+(S)} c_e$. Summing net outflow over all $v \in S$ cancels every internal arc and gives

$$|f| = \sum_{e \in \delta^+(S)} f_e - \sum_{e \in \delta^-(S)} f_e \leq c(S). \quad (1)$$

An especially useful equivalent identity is

$$c(S) - |f| = \sum_{e \in \delta^+(S)} (c_e - f_e) + \sum_{e \in \delta^-(S)} f_e. \quad (2)$$

All terms on the right are nonnegative. This is also the total capacity of the residual arcs leaving S , with each original crossing arc contributing exactly one term.

Theorem 1.1 (Augmenting paths and optimality). For a feasible flow f , the following are equivalent:

1. f is maximum;
2. G_f has no s - t path;
3. $|f| = c(S)$ for some s - t cut S .

Consequently maximum flow value equals minimum cut capacity.

Proof. A residual path permits a strictly positive augmentation, so a maximum flow has no such path. If none exists, define $S = \{v : v \text{ is reachable from } s \text{ in } G_f\}$. Then $s \in S, t \notin S$. For an original arc leaving S , $c_e - f_e = 0$; otherwise its forward residual arc would make its head reachable. For an original arc entering S , $f_e = 0$; otherwise its reverse residual arc would do the same. Equation (2) gives $c(S) = |f|$. Finally, equality certifies optimality of both f and S by (1). A maximum exists because the feasible set is nonempty, closed, and bounded; applying the equivalence to it proves the equality of the two optima, even for real capacities. $\square$

Proof trap: define S by residual reachability. An arbitrary partition at termination need not be minimum; an outgoing residual arc contradicts the head's non-reachability, not necessarily non-reachability of t directly.

1.3 Ford–Fulkerson and integral flows

Algorithm 1: Ford–Fulkerson: arbitrary residual augmenting paths

Input: Network G , capacities $c_e \geq 0$, source s , sink t
 $f_e \leftarrow 0$ for every $e \in E$;
while a residual s - t path exists **do**
 Find a simple s - t path P in G_f ;
 $\Delta \leftarrow \min_{a \in P} c_f(a)$;
 Apply the labelled forward/reverse updates along P by Δ ;
return f and the source-reachable cut in G_f ;

If it terminates, the algorithm is correct by Theorem 1.1. With integer capacities, all flows, residual capacities, and bottlenecks stay integral; each augmentation increases value by at least 1. Thus there are at most $F^* = |f^*|$ augmentations and time $O(m(1 + F^*))$ after discarding isolated vertices. This is *pseudo-polynomial*: a binary capacity C occupies $O(\log C)$ bits, not C bits. Rational capacities also give termination after scaling by a common denominator, without a polynomial iteration bound from this argument. For irrational capacities, unfortunate path choices may cause infinitely many augmentations; arbitrary-path Ford–Fulkerson is not a capacity-independent algorithm.

Integer bad example: $2C$ augmentations instead of two. Use arcs $s \rightarrow a, s \rightarrow b, a \rightarrow t, b \rightarrow t$ of capacity C and $a \rightarrow b$ of capacity 1. Alternate $s \rightarrow a \rightarrow b \rightarrow t$ with $s \rightarrow b \rightarrow a \rightarrow t$, where the second path uses the reverse residual arc of $a \rightarrow b$. Each augmentation has bottleneck 1; every pair increases all four outer flows by 1 and returns the middle flow to 0. There are $2C$ augmentations before reaching value $2C$, exponential in the bit length of C . Taking the two direct paths would require only two augmentations.

Theorem 1.2 (Edgewise integrality). With integer capacities, there exists a maximum flow with $f_e \in \mathbb{Z}$ for every original arc e .

Proof. Start at zero and use Ford–Fulkerson. Every augmentation is integral, and termination follows from the finite integer upper bound on flow value. $\square$

This is an existence theorem, not a statement that every maximum flow is integral. For example, unit arcs $s \rightarrow a, s \rightarrow b, a \rightarrow c, b \rightarrow c, c \rightarrow t$ admit a maximum flow of value 1 that sends $1/2$ along each branch. An integral maximum also exists. An integer *total value* alone is not enough to decode a matching or an indivisible assignment.

1.4 Capacity scaling: $O(m^2(1 + \log C))$

Assume integer capacities and $C = \max_e c_e$ (return zero if E is empty or $C = 0$). For a threshold k , retain only labelled residual arcs of capacity at least k ; call this graph $G_f^{\geq k}$.

Algorithm 2: Capacity scaling: augment only through residual capacities $\geq k$

```

 $f \leftarrow 0; k \leftarrow 2^{\lceil \log_2 C \rceil};$ 
while  $k \geq 1$  do
    while  $G_f^{\geq k}$  has an  $s$ - $t$  path do
        Find an  $s$ - $t$  path  $P$  in  $G_f^{\geq k}$ ;
         $\Delta \leftarrow \min_{a \in P} c_f(a)$ ;
        Apply the labelled updates along  $P$  by  $\Delta$ ;
     $k \leftarrow k/2$ ;
return  $f$ ;

```

Correctness. Feasibility is preserved. At the end of the $k = 1$ phase, integral residual capacities imply every positive residual arc has capacity at least 1. Hence there is no residual s - t path, and f is maximum. **Threshold trap:** starting at $k = C$ with real halving can miss 1. For a path with capacities 3, 1, thresholds 3, 1.5 find no path and then stop, incorrectly returning zero. Powers of two ensure a final phase at 1.

At most $2m$ augmentations per phase. At the end of a k phase, let S be the vertices reachable in $G_f^{\geq k}$. Each residual arc leaving S has capacity less than k . Each original crossing arc contributes once to (2), so

$$F^* - |f| \leq c(S) - |f| < mk \quad (m > 0).$$

At the start of the next phase, whose threshold is $k/2$, the gap is less than $2m(k/2)$. Equivalently, a phase with threshold k starts with gap less than $2mk$. This also holds for the first phase since $F^* \leq \sum_{e \in \delta^+(s)} c_e \leq mC < 2mk$. Every augmentation adds at least k , so a phase has fewer than $2m$ augmentations. The phase need not itself reach the optimum. There are $1 + \lfloor \log_2 C \rfloor$ phases and each path search/update costs $O(m)$ in the usual graph-operation model, giving $O(m^2(1 + \log C))$ time.

1.5 Shortest augmenting paths: $O(nm^2)$

Use BFS in the full positive-residual graph to find an s - t path with the *fewest arcs*; augment by its bottleneck and repeat until BFS fails. This is the standard Edmonds–Karp shortest-augmenting-path rule. Some course materials use different names: remember the path rule and its bound. It works with arbitrary nonnegative real capacities in the exact-arithmetic model; the number of graph operations does not depend on their magnitudes.

Algorithm 3: Shortest augmenting paths (BFS; not arbitrary path choice)

```

 $f \leftarrow 0$ ;
while BFS in  $G_f$  reaches  $t$  from  $s$  do
    Let  $P$  be the shortest residual path given by BFS parent pointers;
     $\Delta \leftarrow \min_{a \in P} c_f(a)$ ;
    Apply the labelled updates along  $P$  by  $\Delta$ ;
return  $f$ ;

```

Lemma 1.3 (Residual distances never decrease). Let $d(v)$ be the BFS distance from s before an augmentation and $d'(v)$ afterwards, with distance ∞ for unreachable vertices. Then $d'(v) \geq d(v)$ for every v .

Proof. If false, take a shortest new path to a vertex whose distance decreased, and its first vertex v with $d'(v) < d(v)$. Its predecessor u satisfies $d'(u) \geq d(u)$ and $d'(v) = d'(u) + 1$. If $u \rightarrow v$ was an old residual arc, then $d(v) \leq d(u) + 1 \leq d'(u) + 1 = d'(v)$, a contradiction. Otherwise it is a newly available reverse of an arc $v \rightarrow u$ on the old shortest augmenting path. Every prefix of that path is shortest, so $d(u) = d(v) + 1$, and $d'(v) = d'(u) + 1 \geq d(u) + 1 = d(v) + 2$, again a contradiction. $\square$

Lemma 1.4 (A residual arc is critical only $O(n)$ times). Call a labelled residual arc *critical* when an augmentation saturates it. Each of the $2m$ possible residual labels is critical at most $O(n)$ times.

Proof. Suppose $a : u \rightarrow v$ is saturated on a shortest path when distances are d . Then $d(v) = d(u) + 1$. Before a can be used and saturated again, positive capacity on its particular label must be restored by an augmentation using its paired reverse label $v \rightarrow u$. At that intervening augmentation, with distances $\hat{d}$, shortest-path choice and monotonicity imply

$$\hat{d}(u) = \hat{d}(v) + 1 \geq d(v) + 1 = d(u) + 2.$$

At the next critical use of a , its tail distance is at least $\hat{d}(u)$. Thus the tail distance increases by at least 2 between critical uses. A finite shortest simple path has at most $n - 1$ arcs, so only $O(n)$ such uses are possible. $\square$

Each augmentation saturates at least one label, so there are $O(nm)$ augmentations. BFS and updating take $O(n + m)$ time, or $O(m)$ after irrelevant isolated vertices are omitted. Thus the standard total is $O(nm^2)$. Finite termination plus Theorem 1.1 proves correctness. This algorithm augments one path per BFS; a blocking-flow algorithm is a different variant.

1.6 Flow decomposition and reduction templates

Decomposition. A nonnegative feasible flow of value $F \geq 0$ decomposes into weighted simple s - t paths of total weight F and directed cycles. To see this, add a labelled artificial arc $t \rightarrow s$ carrying F , making a circulation. Follow positive-flow arcs until a cycle appears and subtract its minimum flow. Conservation prevents getting stuck; each subtraction eliminates an arc, so repeating gives a cycle decomposition. Removing the artificial arc turns precisely its cycles into s - t paths. Integer flows give integer weights, hence F unit paths after splitting weights. Cycles may be discarded without changing value or violating capacity constraints.

Reduction checklist. Specify vertices, arcs, capacities, source/sink, and target flow value. Prove (i) an original solution gives a flow and (ii) an *integral* flow meeting the target decodes into an original solution. State graph size and invoke a polynomial-time flow algorithm. A fractional maximum-flow witness need not decode; choose an integral maximum instead.

Bipartite matching. For a bipartite graph $(L \cup R, E)$, make $s \rightarrow u$ for $u \in L$, $u \rightarrow v$ for $uv \in E$, and $v \rightarrow t$ for $v \in R$, all of capacity 1. A matching of size q gives q unit paths. Conversely, in an integral flow, edges uv carrying 1 form a matching: source/terminal capacity 1 and conservation allow at most one at each endpoint. Their number equals the flow value. Thus maximum matching size is the maximum flow value; matching every left vertex means target $|L|$.

Vertex capacities. To impose throughput b_v on an internal vertex v , replace it by $v_{\text{in}} \rightarrow v_{\text{out}}$ of capacity b_v . Redirect each original $u \rightarrow v$ to $u_{\text{out}} \rightarrow v_{\text{in}}$, retaining its capacity. Incoming/outgoing notation refers to unsplit vertices where appropriate. Conservation forces all throughput across the splitting arc; conversely an original flow respecting b_v extends by putting that throughput on it. Leave terminals unsplit unless their own throughput is constrained.

Multiple sources/sinks and exact supplies. Add a super-source s_0 with $s_0 \rightarrow a_i$ of capacity equal to source i 's supply bound, and arcs $b_j \rightarrow t_0$ for sink receipt bounds. If exact supplies a_i and demands b_j sum to B , target value B forces all these arcs to be saturated and gives the prescribed amounts. For unrestricted auxiliary arcs, use a finite upper bound on all possible flow (for example $1 + \sum_e c_e$), rather than an unspecified infinity.

Edge-disjoint paths. Give every original directed arc capacity 1. A family of q edge-disjoint s - t paths gives a feasible value- q flow. Conversely, decompose an integral maximum flow into unit paths and discard cycles. Capacity 1 prevents any original arc from belonging to two paths. Hence their maximum number equals the minimum number of outgoing arcs in an s - t cut. For undirected edges, insert both orientations of capacity 1. Cancel equal flow on opposite orientations first; this preserves conservation and value. Then decomposition uses each undirected edge at most once across all paths. A cut counts exactly one orientation of each crossing undirected edge.

1.7 Vertex-disjoint paths and Menger's theorem

Theorem 1.5 (Vertex form of Menger). For distinct nonadjacent vertices x, y in an undirected graph, the maximum number p of internally vertex-disjoint x - y paths equals the minimum size q of a set $Z \subseteq V \setminus \{x, y\}$ whose deletion separates x from y .

Proof. Let N be the original number of vertices and choose $M = N + 1$. Split every internal vertex into $v_{\text{in}} \rightarrow v_{\text{out}}$ of capacity 1; keep x, y unsplit. Replace every undirected edge $\{u, v\}$ by $u_{\text{out}} \rightarrow v_{\text{in}}$ and $v_{\text{out}} \rightarrow u_{\text{in}}$, each of capacity M , interpreting unsplit endpoints as themselves. Source is x , sink is y .

Paths correspond to flow. Internally disjoint paths give unit flows without exceeding any splitting capacity, so maximum flow $F^* \geq p$. Conversely, integral decomposition produces F^* unit paths. No two can use the same internal splitting arc, so they decode to internally vertex-disjoint paths. Thus $p = F^*$.

Separators correspond to cuts. Given a separator Z , delete its splitting arcs and take the source-reachable set in the remaining network. The sink is unreachable. Every arc crossing out of this set in the original network must be a deleted splitting arc, so it defines a cut of capacity at most $|Z|$. Hence minimum cut capacity $C^* \leq q$. Because x, y are nonadjacent, deleting all $N - 2$ other vertices separates them; therefore $C^* \leq N - 2 < M$. No minimum cut can contain an outgoing M -capacity arc. Its outgoing arcs are all unit splitting arcs; let Z be their original vertices. Every x - y path must cross that cut and hence one of those splitting arcs, so Z is a separator with $|Z| = C^*$. Thus $q \leq C^*$ as well. Max-flow/min-cut gives $p = F^* = C^* = q$. $\square$

Nonadjacency is needed for this separator formulation: deleting internal vertices cannot destroy a direct edge xy . The same splitting idea gives the directed internally vertex-disjoint path variant when an internal separator exists.

1.8 Layered scheduling with two quota levels

There are workers $i = 1, \dots, r$, periods $j = 1, \dots, k$, and D tasks. Task d belongs to period $p(d)$. Worker i can perform tasks in A_i , may perform at most b_i tasks overall, and at most one per period. Every task needs exactly one worker. Construct the layers

$$s \xrightarrow{b_i} w_i \xrightarrow{1} w_{ij} \xrightarrow{1} d \xrightarrow{1} t, \quad d \in A_i, \quad p(d) = j.$$

There is one w_i per worker, one w_{ij} per worker-period, one vertex per task, and no other arcs. The target is D .

Both directions. Given a feasible schedule, send one unit along the indicated path for each task's assigned worker. Total and period quotas enforce the first two capacities; each task is used once, enforcing the last two. Conservation holds along each path, so the flow has value D . Conversely, an integral flow of value D saturates every task-to-sink arc. At each task, exactly one incoming assignment arc carries 1. Decode that worker as its assignee. Assignment arcs guarantee availability; capacity 1 on $w_i \rightarrow w_{ij}$ enforces the period quota; capacity b_i on $s \rightarrow w_i$ enforces the total quota. Thus a feasible schedule exists iff maximum flow has value D . A fractional assignment is not a schedule, but the integral maximum-flow theorem supplies the required witness.

ILP formulation. Use binary x_{id} only for available pairs $d \in A_i$:

$$\begin{aligned} & \text{minimize} && 0 \\ & \text{subject to} && \sum_{i: d \in A_i} x_{id} = 1 \quad \text{for every task } d, \\ & && \sum_{d \in A_i} x_{id} \leq b_i \quad \text{for every worker } i, \\ & && \sum_{d \in A_i: p(d)=j} x_{id} \leq 1 \quad \text{for every worker-period } (i, j), \\ & && x_{id} \in \{0, 1\}. \end{aligned}$$

Unavailable variables are absent or fixed to zero. A period quota b_{ij} replaces 1 in the second-layer capacity and corresponding ILP constraint. The network has $O(rk + D)$ vertices and $O(r + rk + D +$

$\sum_i |A_i|$) arcs, so one polynomial-time max-flow computation suffices. If an original formulation allows redundant coverage and only imposes upper quotas, discard excess assignments first to get exactly one worker per task.

1.9 Minimum-cut structure and tie-breaking

Theorem 1.6 (Every minimum cut is saturated by every maximum flow). Fix any maximum flow f . For every minimum cut S , all original arcs leaving S satisfy $f_e = c_e$, and all original arcs entering S satisfy $f_e = 0$.

Proof. Since $c(S) = |f|$, the left side of (2) is zero. Every term on its right is nonnegative, so each is zero. $\square$

No opposite crossings. A positive-capacity arc cannot leave one minimum cut's source side and enter another's. Fix the *same* maximum flow for both cuts: the first requires $f_e = c_e > 0$, while the second requires $f_e = 0$, a contradiction. The positive-capacity assumption is necessary for this argument.

Minimum capacity first, then fewest outgoing arcs. For integer capacities, define $c'_e = (m + 1)c_e + 1$ and compute a minimum cut under c' . Its transformed capacity is

$$c'(S) = (m + 1)c(S) + |\delta^+(S)|.$$

Any increase of at least 1 in original cut capacity raises the first term by at least $m + 1$, which exceeds every possible difference in outgoing-arc count (at most m). Therefore a minimum transformed cut first minimizes $c(S)$ and, among ties, minimizes $|\delta^+(S)|$. The capacity encoding grows by only $O(\log(m + 1))$ bits. Integrality is essential to the unit-gap argument; arbitrary real capacities need a separately justified gap bound.

1.10 Optional extension: lower bounds and vertex demands

This construction extends the basic flow reductions. It is useful reference material; the supplied lecture only briefly mentions these extensions. Suppose each arc must satisfy $\ell_e \leq f_e \leq c_e$, and vertices require *net inflow* $\text{in}_f(v) - \text{out}_f(v) = d(v)$. Here $d(v) > 0$ means demand and $d(v) < 0$ means supply. Check $0 \leq \ell_e \leq c_e$ and $\sum_v d(v) = 0$ first. Set $f_e = \ell_e + g_e$, so $0 \leq g_e \leq c_e - \ell_e$, and define

$$d'(v) = d(v) - \sum_{e \in \delta^-(v)} \ell_e + \sum_{e \in \delta^+(v)} \ell_e.$$

Then g must satisfy $\text{in}_g(v) - \text{out}_g(v) = d'(v)$. Add a super-source σ and super-sink τ . For $d'(v) < 0$, add $\sigma \rightarrow v$ of capacity $-d'(v)$; for $d'(v) > 0$, add $v \rightarrow \tau$ of capacity $d'(v)$. Let $B = \sum_{d'(v) > 0} d'(v) = \sum_{d'(v) < 0} -d'(v)$. There is a feasible original flow iff the auxiliary maximum flow has value B , saturating all super-source and super-sink arcs.

Why this works. A valid g extends to such a flow by placing $-d'(v)$ or $d'(v)$ on its auxiliary arc, restoring conservation. Conversely, saturating those arcs and applying conservation at v gives net original inflow $d'(v)$. Deleting auxiliary arcs and adding ℓ recovers a feasible f . Integral data give an integral witness. For lower-bounded s - t flow with unconstrained nonnegative value, add an auxiliary $t \rightarrow s$ arc with a sufficiently large capacity and solve circulation feasibility; its flow records the s - t value. Maximizing that value requires a further augmentation step after feasibility, not just a feasibility test.